MODUL X
PENJADWALAN CPU 1

1. Tujuan Praktikum

1.1. Memahami konsep dasar penjadwalan CPU pada sistem operasi.

1.2. Memahami perbedaan algoritma First Come First Served (FCFS), Shortest Job First (SJF), dan Round Robin (RR).

1.3. Membuat Gantt Chart serta menghitung waiting time dan turnaround time dari proses-proses yang dijadwalkan.

1.4. Membuat, mengkompilasi, dan menjalankan program simulasi penjadwalan CPU FCFS pada lingkungan Linux.

2. Teori

2.1. Pengertian Penjadwalan CPU

Penjadwalan CPU (CPU Scheduling)merupakan mekanisme sistem operasi untuk menentukan proses yang memperoleh kesempatan menggunakan CPU. Pada sistem yang menjalankan banyak proses, penjadwalan diperlukan agar penggunaan CPU berjalan efisien dan proses dapat diselesaikan dengan waktu tunggu yang sesuai.

1. **CPU Utilization**: Persentase waktu CPU digunakan untuk menjalankan proses. Semakin tinggi nilainya, semakin sedikit waktu CPU menganggur.
2. **Throughput**: Jumlah proses yang berhasil diselesaikan dalam satuan waktu tertentu.
3. **Turnaround Time**: Total waktu sejak proses datang hingga selesai dieksekusi.
4. **Waiting Time**: Total waktu yang dihabiskan proses untuk menunggu dalam antrean siap (*ready queue*).
5. **Response Time**: Waktu sejak proses atau permintaan datang hingga pertama kali memperoleh respons.

Secara umum, penjadwalan bertujuan untuk memaksimalkan CPU utilization dan throughput serta meminimalkan turnaround time, waiting time, dan response time.

2.2. Istilah dan Perhitungan Penjadwalan CPU

Beberapa istilah yang digunakan dalam simulasi penjadwalan CPU adalah:

- **Arrival Time (AT)**: Waktu kedatangan proses.
- **Burst Time (BT)**: Lama waktu CPU yang dibutuhkan proses untuk menyelesaikan eksekusinya.
- **Completion Time (CT)**: Waktu ketika proses selesai dieksekusi.
- **Turnaround Time (TAT)**: Total waktu yang dibutuhkan sejak proses datang hingga selesai.
- **Waiting Time (WT)**: Total waktu proses menunggu sebelum atau di antara giliran eksekusinya.

Rumus yang digunakan:

- Turnaround Time: $TAT = CT - AT$
- Waiting Time: $WT = TAT - BT$

Untuk menghitung waktu tunggu rata-rata dari sejumlah proses (Average Waiting Time): $AWT = \frac{\Sigma WT}{n}$

Keterangan:

- ΣWT = jumlah waiting time seluruh proses
- n = jumlah proses

Pada simulasi sederhana tanpa operasi I/O, rumus $WT = TAT - BT$ dapat digunakan untuk menghitung total waktu tunggu proses, termasuk pada Round Robin.

2.3. Algoritma Penjadwalan CPU

2.3.1. First Come First Served (FCFS)

FCFS merupakan algoritma penjadwalan yang mengeksekusi proses berdasarkan urutan kedatangannya. Proses yang datang lebih dahulu memperoleh CPU lebih dahulu. FCFS menggunakan prinsip antrean FIFO (*First In, First Out*) dan bersifat non-preemptive, yaitu proses yang sedang

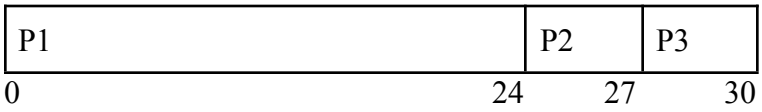
berjalan tetap menggunakan CPU hingga selesai. Kelemahan FCFS adalah *convoy effect*, yaitu proses-proses pendek harus menunggu lama apabila berada di belakang proses yang membutuhkan waktu eksekusi panjang.

Contoh:

Ada 3 proses P1, P2, P3 yang datang dengan lama waktu kerja CPU (CPU Burst Time) masing-masing sebagai berikut:

Proses	Burst-Time
P1	24
P2	3
P3	3

Dengan menggunakan algoritma FCFS, maka dapat digambarkan Gantt Chart-nya sebagai berikut:



2.3.2. Shortest Job First (SJF)

SJF merupakan algoritma yang memprioritaskan proses dengan burst time paling pendek di antara proses yang sudah tersedia dalam antrean siap. Pada praktikum ini digunakan SJF non-preemptive, sehingga proses yang sudah mulai dieksekusi akan berjalan sampai selesai. Apabila terdapat beberapa proses dengan burst time yang sama, urutan kedatangan digunakan sebagai penentu. SJF dapat menghasilkan rata-rata waiting time yang lebih rendah dibandingkan FCFS pada kumpulan proses tertentu.

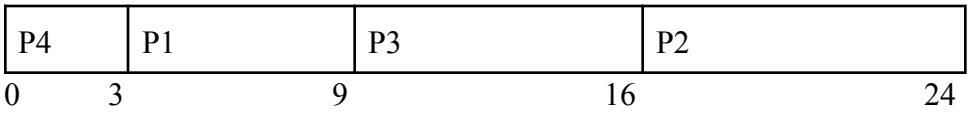
Contoh:

Ada 4 proses P1, P2, P3, P4 yang datang dengan lama waktu kerja CPU (CPU Burst Time) masing-masing sebagai berikut:

Proses	Burst-Time
P1	6

P2	8
P3	7
P4	3

Dengan menggunakan algoritma SJF, maka dapat digambarkan Gantt Chart-nya sebagai berikut:



2.3.3. Round Robin (RR)

Round Robin merupakan algoritma penjadwalan yang memberikan CPU kepada setiap proses selama jatah waktu tertentu, yang disebut *time quantum*. Jika proses belum selesai ketika jatah waktunya habis, proses tersebut dikembalikan ke bagian belakang antrean untuk memperoleh giliran berikutnya. Round Robin memberikan kesempatan CPU secara bergantian kepada proses-proses yang menunggu. Besarnya time quantum memengaruhi frekuensi pergantian proses dan waktu respons sistem.

Contoh:

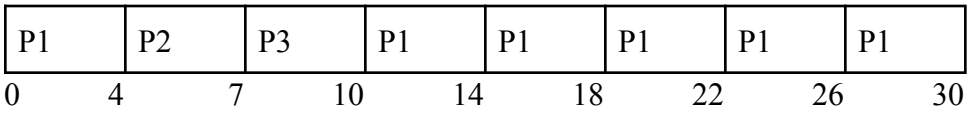
Ada 3 proses P1, P2, P3 yang datang dengan lama waktu kerja CPU (CPU Burst Time) masing-masing sebagai berikut:

Proses	**Burst-Time**
P1	24
P2	3
P3	3

Time quantum: 4 milidetik

- P1 mendapatkan 34 milidetik pertama
- 20 milidetik berikutnya akan disela P2 dan P3

Dengan menggunakan algoritma RR, maka dapat digambarkan Gantt Chart-nya sebagai berikut:



3. Kegiatan

3.1. Kegiatan 1: Simulasi Algoritma Penjadwalan CPU

Langkah-langkah:

1. Perhatikan tabel proses berikut:

Proses	Burst-Time
P1	10
P2	1
P3	2
P4	1
P5	5

2. Gunakan ketentuan berikut:
    - Semua proses datang pada waktu 0.
    - Urutan awal proses adalah P1, P2, P3, P4, P5.
    - Time quantum untuk Round Robin adalah 3 satuan waktu.
    - Jika terdapat burst time yang sama pada SJF, gunakan urutan awal sebagai penentu.
3. Buat Gantt Chart untuk masing-masing algoritma:
    - FCFS.
    - SJF non-preemptive.
    - Round Robin dengan time quantum = 3.
4. Hitung waiting time setiap proses dan waiting time rata-rata untuk masing-masing algoritma!
5. Catat hasilnya pada tabel berikut:

Algoritma	Urutan Eksekusi	Average Waiting Time

FCFS		
SJF non-preemptive		
Round Robin (q = 3)		

Analisis Kegiatan 1

1. Algoritma manakah yang menghasilkan rata-rata waiting time paling rendah pada data tersebut?
2. Mengapa FCFS dapat menyebabkan *convoy effect*?
3. Apa pengaruh perubahan time quantum terhadap pelaksanaan algoritma Round Robin?

3.2. Kegiatan 2: Implementasi Simulasi FCFS pada Linux

Pada kegiatan ini, mahasiswa membuat dan menjalankan program bahasa C untuk menyimulasikan penjadwalan CPU menggunakan algoritma FCFS.

Langkah-langkah:

1. Buka terminal GNU/Linux.

   Buat direktori kerja dan masuk ke dalamnya menggunakan perintah:

   ```
   mkdir -p praktikum_cpu1
   cd praktikum_cpu1
   ```

2. Buat file program.

   Gunakan editor teks yang tersedia, misalnya Nano:

   ```
   nano fcfs.c
   ```

   Ketik atau salin kode program berikut ke dalam file `fcfs.c`:

```
#include <stdio.h>
#include <string.h>
     int main(void) {
          int n, at[100], bt[100], wt[100];
          int tat[100], ct[100], start[100];
          int i, j, temp, time = 0;
          int totalWT = 0, totalTAT = 0;
          char name[100][20], tempName[20];
```

```
    printf("===  Simulasi  Penjadwalan  CPU
FCFS ===\n");
    printf("Jumlah proses (1-100): ");

    if (scanf("%d", &n) != 1 || n < 1 || n
> 100) {
        printf("Jumlah    proses    tidak
    valid.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("\nProses ke-%d\n", i + 1);

        printf("Nama proses: ");
        scanf("%19s", name[i]);

        printf("Arrival time: ");
        if  (scanf("%d",  &at[i])  !=  1  ||
    at[i] < 0) {
            printf("Arrival   time   tidak
        valid.\n");
            return 1;
        }

        printf("Burst time: ");
        if  (scanf("%d",  &bt[i])  !=  1  ||
        bt[i] <= 0) {
```

```
                    printf("Burst    time    tidak
               valid.\n");
                    return 1;
               }
          }

          /*  Urutkan  proses  berdasarkan  arrival
     time. */
          for (i = 0; i < n - 1; i++) {
               for (j = 0; j < n - i - 1; j++) {
                    if (at[j] > at[j + 1]) {
                         temp = at[j];
                         at[j] = at[j + 1];
                         at[j + 1] = temp;

                         temp = bt[j];
                         bt[j] = bt[j + 1];
                         bt[j + 1] = temp;

                         strcpy(tempName,
                    name[j]);
                         strcpy(name[j],  name[j  +
                    1]);
                         strcpy(name[j    +    1],
                    tempName);
                    }
               }
          }

          printf("\nTabel Hasil FCFS\n");
```

```
printf("Proses\tAT\tBT\tWT\tCT\tTAT\n")
;

for (i = 0; i < n; i++) {
     if (time < at[i])
          time = at[i];

     start[i] = time;
     wt[i] = start[i] - at[i];
     ct[i] = start[i] + bt[i];
     tat[i] = ct[i] - at[i];

     time = ct[i];
     totalWT += wt[i];
     totalTAT += tat[i];
     printf("%s\t%d\t%d\t%d\t%d\t%d\n",
          name[i], at[i], bt[i],
          wt[i], ct[i], tat[i]);
}

printf("\nGantt Chart\n");
printf("0");

if (start[0] > 0)
     printf("    --    Idle    --    %d",
start[0]);

for (i = 0; i < n; i++) {
     if (i > 0 && start[i] > ct[i - 1])
```

```
                    printf("  --  Idle  --  %d",
                    start[i]);

                printf("  --  %s  --  %d", name[i],
            ct[i]);
            }

            printf("\n\nRata-rata  Waiting  Time  =
            %.2f\n",
                (double) totalWT / n);

            printf("Rata-rata  Turnaround  Time  =
        %.2f\n",
                (double) totalTAT / n);

            return 0;
        }
```

3. Simpan file program.

   Pada Nano, tekan `Ctrl + O`, kemudian `Enter` untuk menyimpan. Tekan `Ctrl + X` untuk keluar dari editor.

4. Kompilasi program.

   Jalankan perintah berikut pada terminal:

   ```
   gcc -Wall -Wextra fcfs.c -o fcfs
   ```

   Apabila tidak terdapat kesalahan kompilasi, program akan menghasilkan file executable bernama `fcfs`.

5. Jalankan program.

   ```
   ./fcfs
   ```

   Masukkan data pengujian berikut ketika program meminta input.

Nama Proses	Arrival Time (AT)	Burst Time (BT)

P1	0	3
P2	1	2
P3	2	1

6. Catat hasil pengamatan.

Parameter Pengamatan	Hasil
Urutan eksekusi proses	
Waiting time P1	
Waiting time P2	
Waiting time P3	
Turnaround time P1	
Turnaround time P2	
Turnaround time P3	
Average Waiting Time	
Average Turnaround Time	

Analisis Kegiatan 2

1. Mengapa P2 harus menunggu meskipun proses tersebut datang sebelum P1 selesai dieksekusi?
2. Apa fungsi arrival time dan burst time dalam program tersebut?
3. Bagaimana urutan eksekusi FCFS ditentukan apabila waktu kedatangan beberapa proses berbeda?

4. Rubrik Penilaian

Jenis Penilaian	Poin Maksimal
Kegiatan 1: Ketepatan Gantt Chart dan perhitungan penjadwalan	30
Kegiatan 2: Keberhasilan kompilasi, eksekusi, dan ketepatan hasil program	40

Analisis dan jawaban pertanyaan	20
Format & Kerapian Lembar Kerja (Selesai pada Sesi Praktikum)	10
Total	**100**

5. Referensi

Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*.

Modul Praktikum Sistem Operasi: Penjadwalan CPU 1, Laboratorium Sistem Komputer & Jaringan.